BugBoard26

Documentazione finale

Requisiti, design e testing

Progetto di Ingegneria del Software
A.A. 2025/2026 – Canale 2

Componenti	Valerio Porcile – N86004517
	Emanuel Giuseppe Lupoli – N86004280
	Emanuele Martucci – N86004084

Modalita'	Tradizionale
Versione	1.6 – relazione tecnica
Data di riferimento	19 ottobre 2026
Codice descritto (commit Git)	`b95d008`

Introduzione

Questo documento riunisce la specifica dei requisiti, il design del sistema e le attivita' di testing in capitoli distinti, secondo la sezione 8 della traccia ufficiale v1.1. Riprende il materiale dei due homework e lo confronta con il codice disponibile nel repository.

La relazione presenta il perimetro assegnato, i casi d'uso, le scelte architetturali e Object-Oriented, il modello dei dati e le verifiche svolte. Il capitolo sul testing distingue test automatici, collaudo del sistema, analisi di qualita' e valutazione dell'usabilita'. Il capitolo conclusivo espone i limiti del prodotto e della valutazione.

La revisione del codice descritta e' `b95d008`. I risultati delle verifiche mantengono il proprio riferimento storico: il collaudo del 16 ottobre riguarda `e139951`; l'audit del 19 ottobre usa la base `6aa175a` con correzioni locali, mentre la successiva review frontend usa `06ae1b0` con modifiche locali. I report conservano stato Git e impronte dei sorgenti per identificare gli esemplari analizzati. La revisione editoriale non costituisce una nuova esecuzione dei test sul commit di copertina.

Repository: https://github.com/Valeriop03/BugBoard26. La traccia di riferimento e' il file `Progetto-SWE-2025-2026-canale2-v1.1.pdf` nella radice. I sorgenti dei homework sono conservati come materiale precedente. I percorsi relativi citati nella relazione si riferiscono alla cartella `Documentazione/Documentazionefinale/`, salvo diversa indicazione.

Indice

Introduzione **i**

1 Specifica dei requisiti **1**
1.1 Scopo e perimetro . . . 1
1.2 Glossario e utenti target . . . 2
1.3 Requisiti funzionali . . . 2
1.4 Requisiti non funzionali e vincoli . . . 3
1.5 Modello dei casi d'uso . . . 3
1.6 UC-01 – Segnalare una issue . . . 5
1.7 UC-02 – Risolvere una issue assegnata . . . 6
1.8 UC-03 – Contrassegnare un duplicato . . . 7

2 Design del sistema e del software **8**
2.1 Architettura e scelte tecnologiche . . . 8
2.2 Organizzazione Object-Oriented . . . 10
2.3 Persistenza . . . 13
2.4 Autenticazione e autorizzazione . . . 15
2.5 Ricerca, suggerimento ed export . . . 15
2.6 Immagini, archivio e duplicati . . . 15
2.7 Diagrammi comportamentali . . . 16
2.8 Interfacce e avvio . . . 19
2.9 Versionamento e artefatti . . . 20

3 Testing e valutazione dell'usabilita' **21**
3.1 Strategia e livelli di verifica . . . 21
3.2 Tre metodi di dominio e classi di equivalenza . . . 21
3.2.1 Classi coperte e limiti della strategia . . . 21
3.3 Test plan – Risoluzione di una issue . . . 22
3.3.1 Ambiente, criteri e raccolta delle evidenze . . . 22
3.4 Collaudo frontend reale – 16 ottobre 2026 . . . 24
3.5 Esecuzione ripetibile e qualita' backend . . . 25
3.6 Valutazione dell'usabilita' – review e sessioni . . . 26
3.6.1 Review tramite checklist . . . 26
3.6.2 Sessioni con utenti e questionario . . . 26
3.6.3 Risultati e limiti delle tre sessioni . . . 27

4 Limiti e conclusioni **28**
4.1 Limiti della valutazione . . . 28
4.2 Provenienza dei diagrammi e verifica CASE . . . 28
4.3 Perimetro del prodotto . . . 28
4.4 Conclusioni . . . 28

Capitolo 1

Specifica dei requisiti

1.1 Scopo e perimetro

BugBoard26 permette a un gruppo di lavoro di segnalare e consultare issue, seguirne lo stato e comunicare la risoluzione al segnalatore. Il progetto e' una web app distribuita con frontend e backend indipendenti. Le funzionalita' assegnate sono esclusivamente quelle riportate sotto.

N. traccia	Funzionalita' assegnata
1	Login email/password, amministratore iniziale e creazione delle utenze da parte di ADMIN.
2	Creazione issue con titolo, descrizione, tipo, priorita' e immagine opzionali; stato iniziale TODO.
3	Lista riepilogativa, filtri e ordinamento.
6	Cambio stato da parte dell'assegnatario; notifica al segnalatore alla risoluzione.
8	Esportazione dell'elenco in formato condivisibile, realizzata con CSV.
11	Ricerca per parole chiave.
13	Archiviazione riservata ad ADMIN e consultazione in una pagina dedicata.
14	Suggerimento dell'assegnatario in base al carico di lavoro corrente.
15	Utenze READONLY create dagli amministratori per la consultazione.
16	Contrassegno duplicato riservato ad ADMIN e chiusura logica immediata.

Non sono incluse gestione dei commenti, dashboard, tag, cronologia, report mensili o scadenze. Il campo assegnatario e' necessario per il cambio stato e il suggerimento, ma non introduce il flusso separato di assegnazione e relativa notifica della funzionalita' 4, non assegnata. La notifica di risoluzione e' interna all'applicazione, non un'email.

1.2 Glossario e utenti target

Termine	Significato
Issue	Segnalazione di tipo QUESTION, BUG, DOCUMENTATION o FEATURE.
Segnalatore	Utente che ha creato la issue.
Assegnatario	Utente indicato nel campo `AssignedToId`.
Archiviata	Issue esclusa dalla lista principale, ma conservata e consultabile.
Duplicata	Issue chiusa logicamente come DUPLICATE e collegata a un originale.
Notifica	Messaggio persistente per un singolo destinatario, con stato di lettura.

Gli utenti target sono membri di un team di sviluppo e referenti esterni. I primi devono registrare e consultare segnalazioni durante il lavoro; i secondi devono seguirne l'avanzamento senza modificare le issue. Non e' previsto l'accesso anonimo ai dati del progetto.

Attore	Responsabilita'
ADMIN	Crea utenti e issue, consulta i dati, cambia stato anche senza assegnazione, archivia e marca duplicati.
USER	Crea e consulta issue; cambia stato soltanto se assegnatario.
READONLY	Consulta issue, dettagli, immagini e archivio; usa ricerca, filtri ed export senza modificare issue.

Tutti i ruoli autenticati possono consultare il suggerimento assegnatario e le proprie notifiche.

Interpretazione adottata per RF-17: READONLY puo' consultare le proprie notifiche, ma non segnarle come lette. Questa scelta applica il divieto di scrittura anche allo stato di lettura. Il contratto e' implementato e verificato con test HTTP e browser: GET proprie 200, PATCH read 403 senza scritture e nessun comando UI, anche per notifiche gia' lette.

1.3 Requisiti funzionali

Gli identificativi seguenti sono mantenuti dalla specifica dei requisiti; non coincidono con i numeri delle funzionalita' nella traccia.

ID	Requisito
RF-01	Login tramite email e password.
RF-02	Account amministratore iniziale gia' attivo.
RF-03	Creazione utenti da ADMIN con email, password e ruolo.
RF-04	Creazione issue da utente autenticato non READONLY con titolo e descrizione.
RF-05	Tipo della issue, priorita' opzionale e immagine opzionale.
RF-06	Stato iniziale TODO.
RF-07	Lista delle issue non archiviate.
RF-08	Filtri e ordinamento per criteri rilevanti.

ID	Requisito
RF-09	Ricerca per parole chiave su titolo e descrizione.
RF-10	Cambio stato da parte dell'assegnatario; ADMIN ha accesso completo.
RF-11	Notifica al segnalatore quando la issue passa a RESOLVED.
RF-12	Esportazione dell'elenco in CSV.
RF-13	Archiviazione riservata ad ADMIN.
RF-14	Esclusione degli archiviati dalla lista principale.
RF-15	Consultazione delle issue archiviate in una pagina dedicata.
RF-16	Suggerimento assegnatario basato sul carico corrente.
RF-17	READONLY visualizza issue e dettagli senza modificare dati.
RF-18	ADMIN puo' contrassegnare una issue come duplicata di un'altra.
RF-19	Chiusura immediata della issue contrassegnata come duplicata.

1.4 Requisiti non funzionali e vincoli

ID	Requisito
RNF-01	Password memorizzate tramite hashing, non in chiaro.
RNF-02	Verifica dei permessi nelle API, non soltanto nell'interfaccia.
RNF-03	Frontend e backend separati; comunicazione esclusivamente via API REST.
RNF-04	Stato persistente gestito centralmente dal backend e dal database.
RNF-05	Interfaccia leggibile, con operazioni disponibili coerenti con il ruolo.
RNF-06	Lista consultabile fluidamente con i dati dimostrativi; nessuna prestazione su larga scala dichiarata senza misure.
RNF-07	Separazione tra controller, servizi, modelli e accesso ai dati.
RNF-08	Avvio locale ripetibile seguendo il README.

Il codice usa linguaggi Object-Oriented e non delega logica e persistenza a servizi MBaaS. La traccia richiede strumenti CASE per il design. Il deployment su cloud e' auspicato, ma non obbligatorio: non costituisce una funzionalita' mancante del progetto.

1.5 Modello dei casi d'uso

Il diagramma rappresenta i ruoli e le funzionalita' assegnate. Il login precede le operazioni protette. La notifica e' legata alla risoluzione; la chiusura del duplicato non richiede un comando separato.

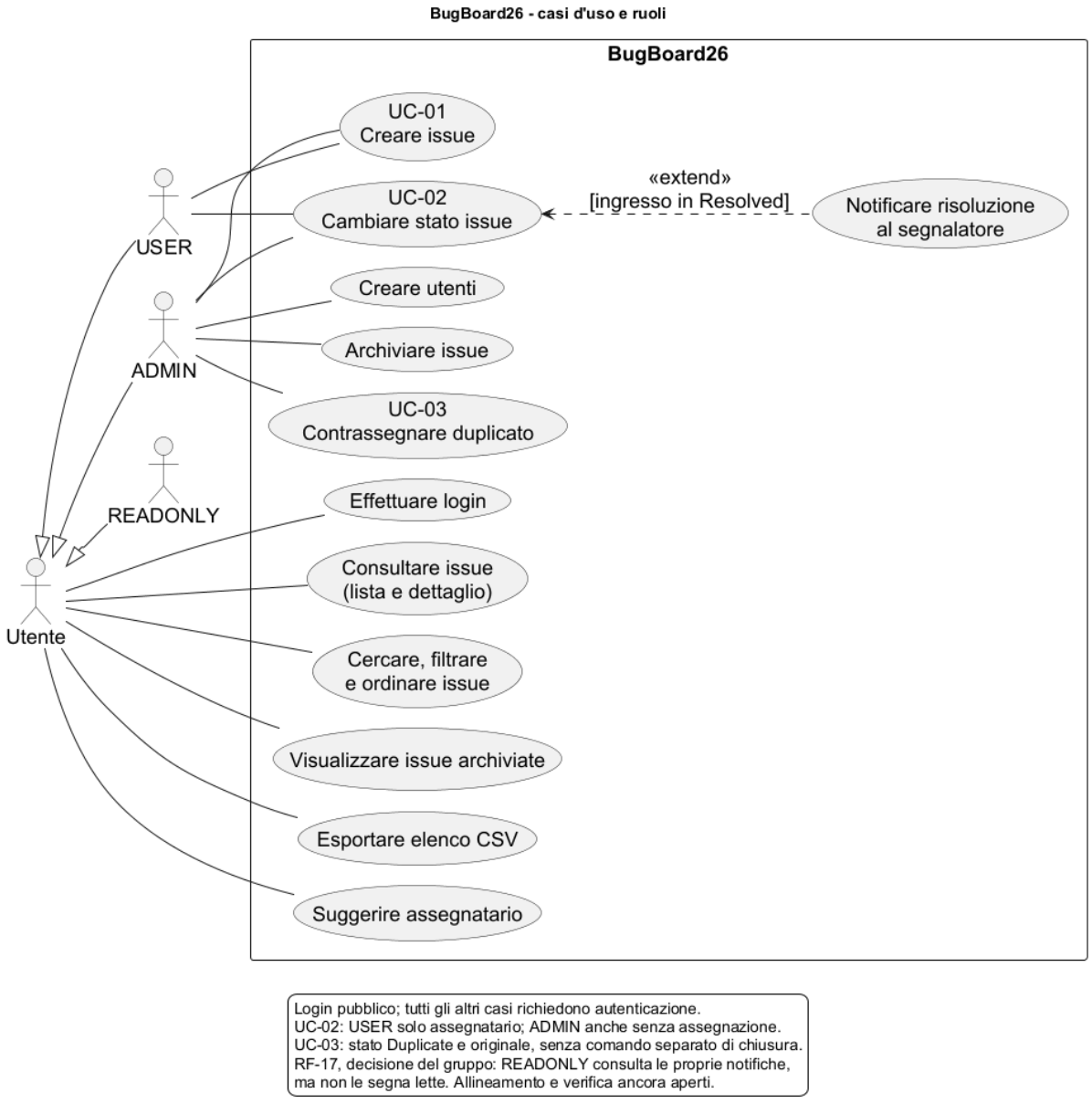


Figura 1.1: Diagramma dei casi d'uso.

1.6 UC-01 – Segnalare una issue

Campo	Descrizione
Attore e scopo	USER oppure ADMIN registra una nuova segnalazione.
Precondizioni	Account attivo e autenticato, non READONLY.
Postcondizione	Issue persistita in TODO e consultabile nella lista e nel dettaglio.
Scenario principale	1. L'utente apre il form. 2. Inserisce titolo, descrizione e tipo; puo' scegliere priorita' e immagine. 3. Il sistema mostra il suggerimento assegnatario. 4. Valida i dati e crea la issue. 5. Se selezionata, carica l'immagine. 6. Apre il dettaglio della issue creata.
Alternative	Dati mancanti/non validi: nessuna creazione. Suggerimento indisponibile: creazione senza assegnatario. Upload fallito dopo creazione: issue conservata, possibilita' di riprovare solo l'upload o aprire il dettaglio senza immagine.
Requisiti	RF-04, RF-05, RF-06, RF-07, RF-16.

Il form richiede almeno tre caratteri dopo la rimozione degli spazi esterni per titolo e descrizione; i limiti massimi sono rispettivamente 200 e 4000 caratteri. Il suggerimento viene usato come assegnatario dal form; non e' una nuova API di assegnazione. Se indicato, l'assegnatario deve essere attivo e non READONLY. La creazione JSON e l'upload multipart sono due richieste distinte.

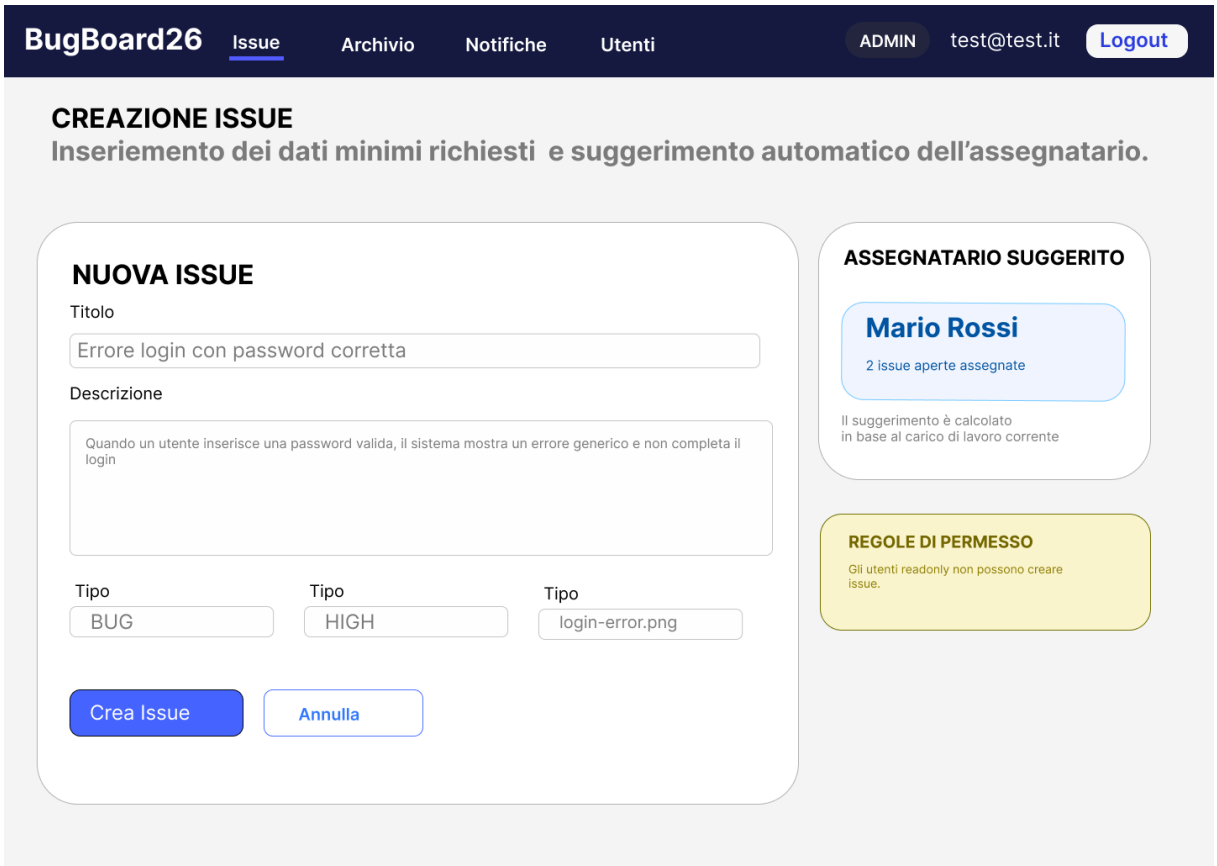


Figura 1.2: Mock-up di creazione: riferimento progettuale, non screenshot del collaudo.

1.7 UC-02 – Risolvere una issue assegnata

Campo	Descrizione
Attore e scopo	USER assegnatario oppure ADMIN aggiorna l'avanzamento della issue.
Precondizioni	Account attivo e autenticato; issue esistente e non contrassegnata come duplicata.
Postcondizioni	Nuovo stato salvato; alla risoluzione sono valorizzati data di risoluzione e notifica non letta per il segnalatore.
Scenario principale	1. L'utente apre il dettaglio. 2. Seleziona RESOLVED. 3. Il backend verifica ruolo e assegnazione. 4. Aggiorna stato e date, crea la notifica e salva. 5. Il frontend mostra il risultato aggiornato.
Alternative	USER non assegnatario o READONLY: 403. Issue inesistente: 404. Stato gia' corrente: risposta senza nuova notifica o nuove date. Issue duplicata: 409. Token non valido: 401.
Requisiti	RF-10, RF-11, RF-17, RNF-02.

Non e' imposta una matrice aggiuntiva di transizioni. Il passaggio da RESOLVED a un altro stato azzera la data di risoluzione; una risoluzione successiva genera una nuova notifica. Le notifiche precedenti restano.

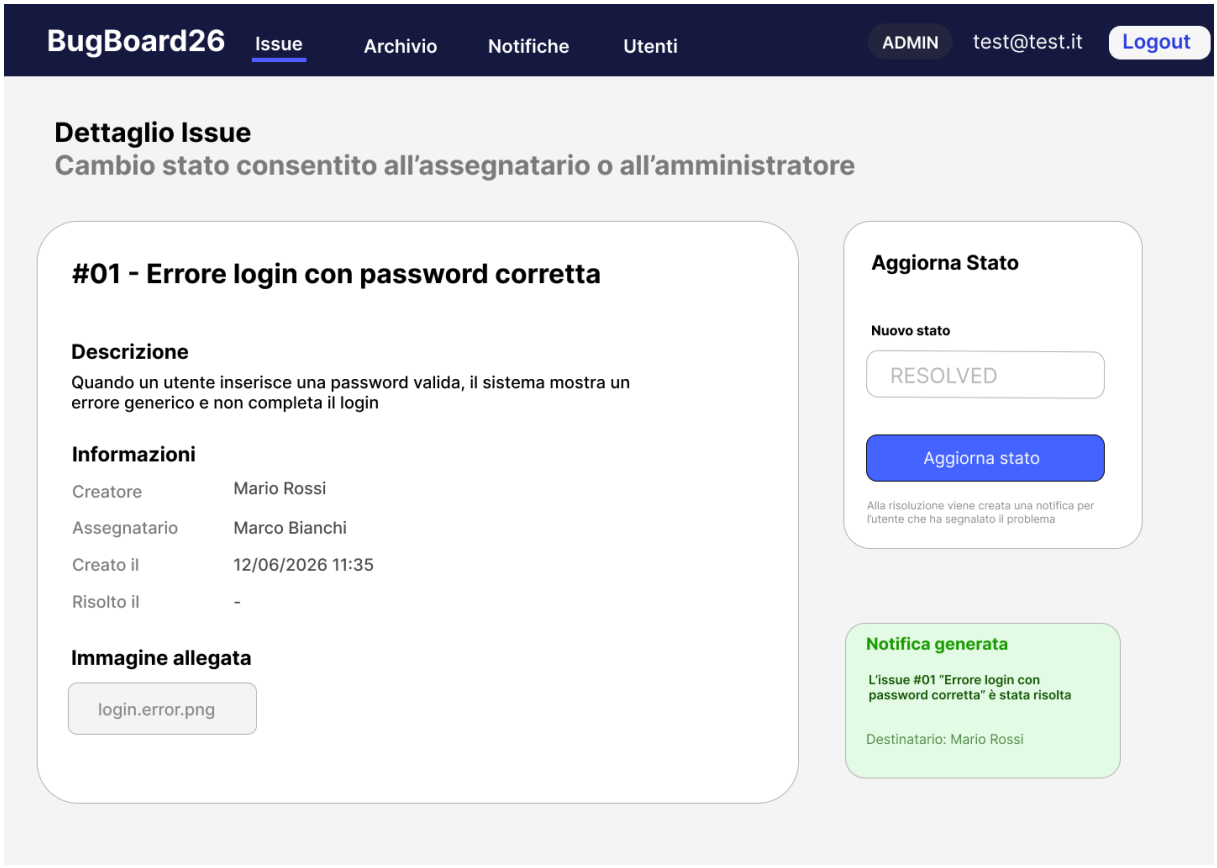


Figura 1.3: Mock-up del dettaglio e del cambio stato.

1.8 UC-03 – Contrassegnare un duplicato

Campo	Descrizione
Attore e scopo	ADMIN collega una segnalazione duplicata all'originale.
Precondizioni	Autenticazione ADMIN; entrambe le issue esistono e sono distinte.
Postcondizioni	Stato DUPLICATE, riferimento all'originale salvato, `ResolvedAt` nullo.
Scenario principale	1. Apre il dettaglio del duplicato. 2. Inserisce l'ID dell'originale. 3. Il backend verifica i riferimenti. 4. Salva stato e collegamento. 5. Il frontend aggiorna il dettaglio.
Alternative	Originale assente o autoreferenza: 400. Issue corrente assente: 404. Ruolo non ADMIN: 403.
Requisiti	RF-18, RF-19, RNF-02.

DUPLICATE rappresenta la chiusura logica, non l'eliminazione. Il percorso di cambio stato non permette di impostarlo direttamente: occorre l'API dedicata con il riferimento all'originale.

Capitolo 2

Design del sistema e del software

2.1 Architettura e scelte tecnologiche

Il frontend Angular presenta i dati e invia richieste HTTP. ASP.NET Core Web API espone le operazioni, controlla i permessi e gestisce la persistenza tramite Entity Framework Core e PostgreSQL. Il backend funziona anche senza frontend; nessuna pagina Angular accede direttamente al database o alla directory delle immagini.

Angular e' usato per routing, form e servizi HTTP. ASP.NET Core permette di organizzare endpoint, dependency injection e autorizzazione nello stesso progetto. PostgreSQL viene avviato in Docker per ridurre le differenze di configurazione fra i tre PC. Il progetto backend usa .NET 10, il frontend Angular 19; le versioni effettive dei pacchetti sono nei rispettivi file di progetto e lockfile.

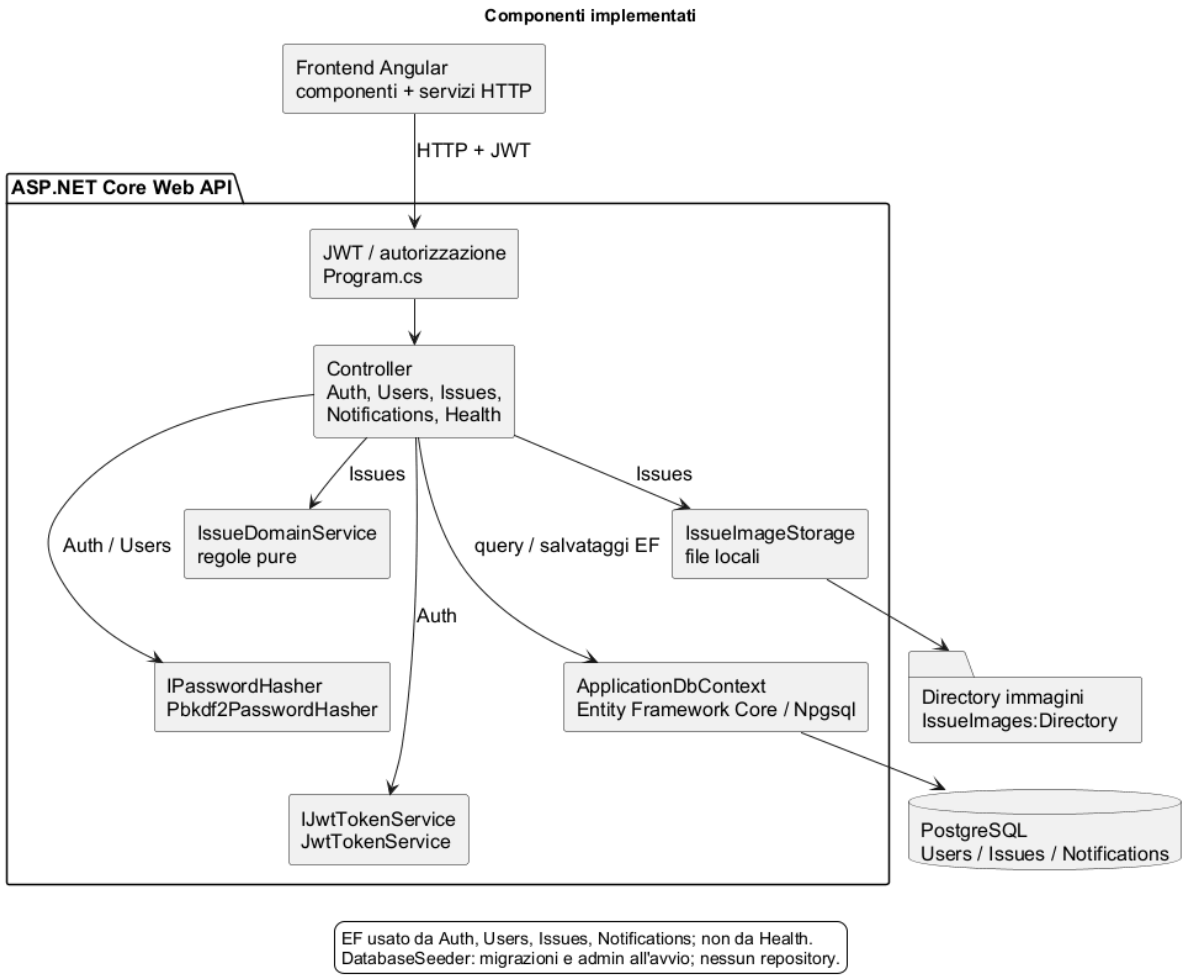


Figura 2.1: Componenti e comunicazione del sistema.

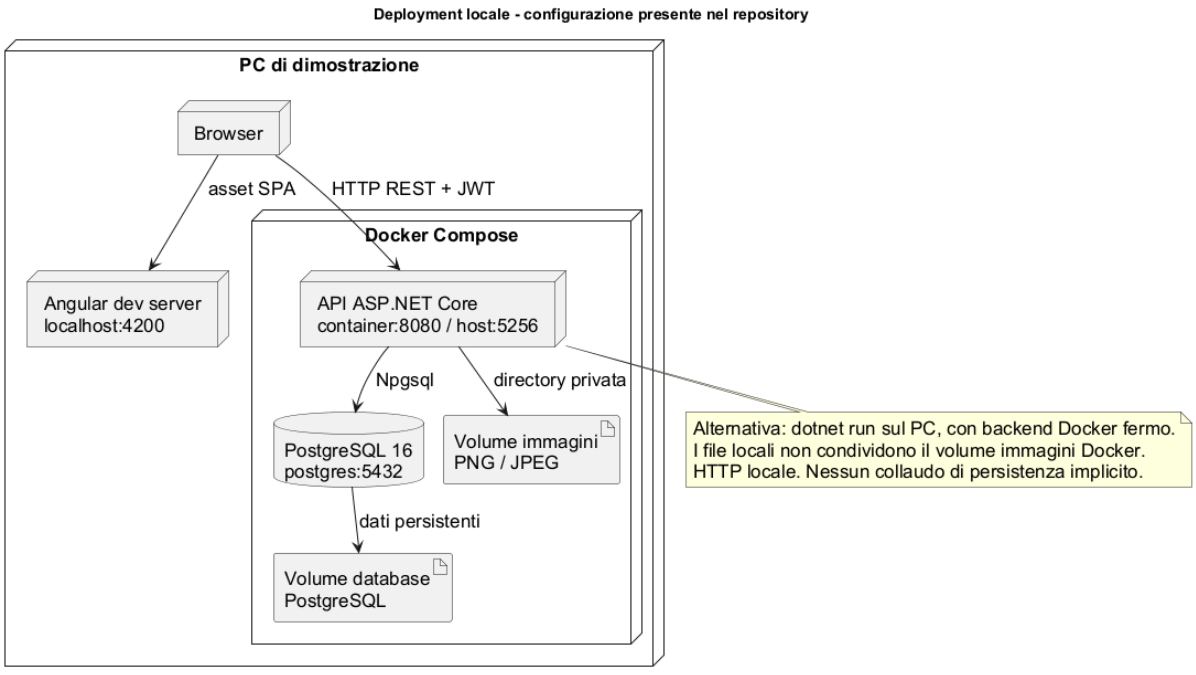


Figura 2.2: Deployment locale con frontend, API e database.

2.2 Organizzazione Object-Oriented

Elemento	Responsabilita'
Controller	Gestiscono richieste e risposte HTTP, autorizzazioni e coordinamento delle operazioni.
`IssueDomainService`	Logiche semplici di ricerca in memoria, permessi sullo stato e suggerimento assegnatario.
`ApplicationDbContext`	Mapping, relazioni e accesso a PostgreSQL con Entity Framework Core.
`User`, `Issue`, `Notification`	Entita' del dominio con proprieta' e relazioni.
`IPasswordHasher`, `IJwtTokenService`	Contratti per hashing password e generazione dei token; implementazioni iniettate nei controller.
`IssueImageStorage`	Validazione e lettura/scrittura delle immagini in una directory privata.
DTO di richiesta/risposta	Separano i dati delle API dalle entita' persistenti; non espongono password hash o percorsi fisici.

La separazione rende testabili le logiche senza HTTP e consente di cambiare la presentazione mantenendo le API. Le interfacce di autenticazione separano il contratto dall'implementazione. Non sono introdotti repository generici o livelli enterprise: per questo perimetro il DbContext e i servizi presenti sono sufficienti. Alcune query e operazioni sono nei controller; non si dichiara una separazione assoluta di tutta la logica.

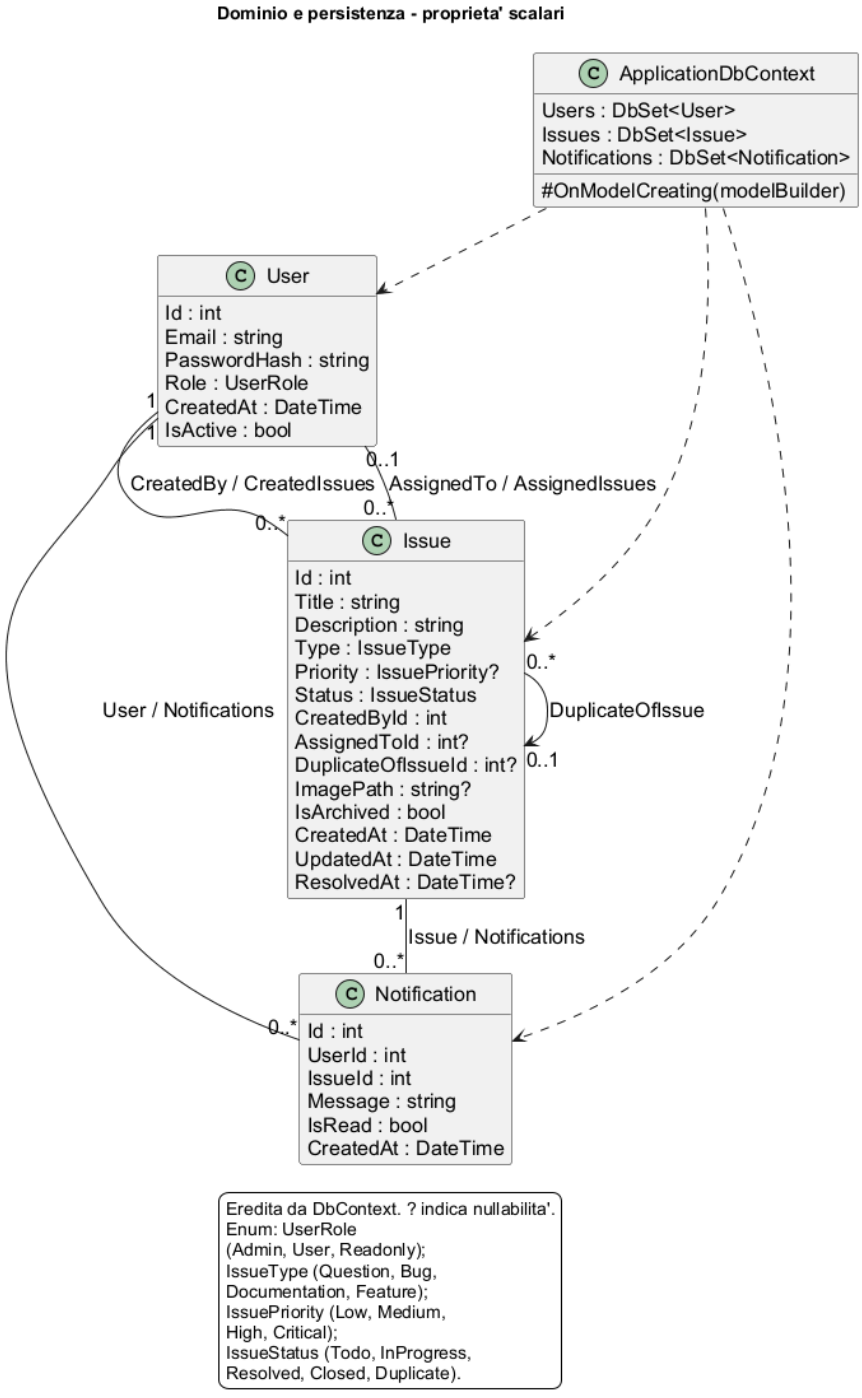


Figura 2.3: Classi di dominio, persistenza e servizi.

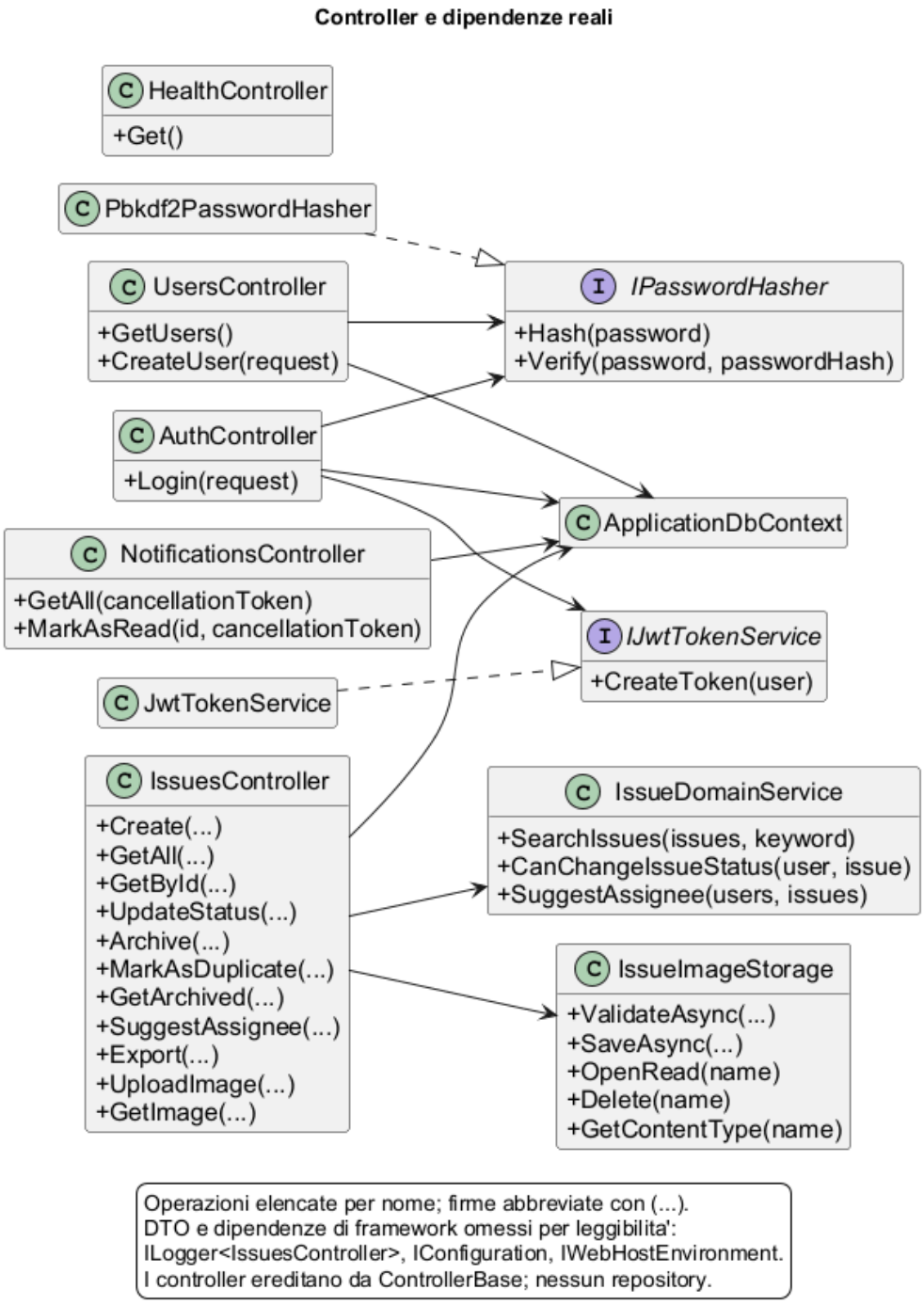


Figura 2.4: Controller e dipendenze.

2.3 Persistenza

Gli identificativi sono interi. Gli enum sono memorizzati come stringhe; le date applicative sono prodotte in UTC.

Entita'	Campi principali
User	Id, Email, PasswordHash, Role, CreatedAt, IsActive. Email univoca; collezioni di issue create/assegnate e notifiche.
Issue	Id, Title, Description, Type, Priority nullable, Status, CreatedById, AssignedToId nullable, DuplicateOfIssueId nullable, ImagePath nullable, IsArchived, CreatedAt, UpdatedAt, ResolvedAt nullable.
Notification	Id, UserId, IssueId, Message, IsRead, CreatedAt.

Ogni issue ha un segnalatore e puo' avere un assegnatario. La relazione con l'originale del duplicato e' un autoriferimento opzionale. Le notifiche appartengono a un destinatario e a una issue. Il mapping impedisce la cancellazione del segnalatore o dell'originale referenziato; la cancellazione dell'assegnatario azzera il riferimento. Non sono esposte nuove funzioni di cancellazione per sfruttare queste regole.

I byte delle immagini non sono in PostgreSQL: il database conserva un nome gestito dal backend. In Docker file e database sono mantenuti in due volumi distinti. Il seed applica le migrazioni e crea l'ADMIN se non esiste gia' un amministratore. Non reimposta la password di account esistenti.

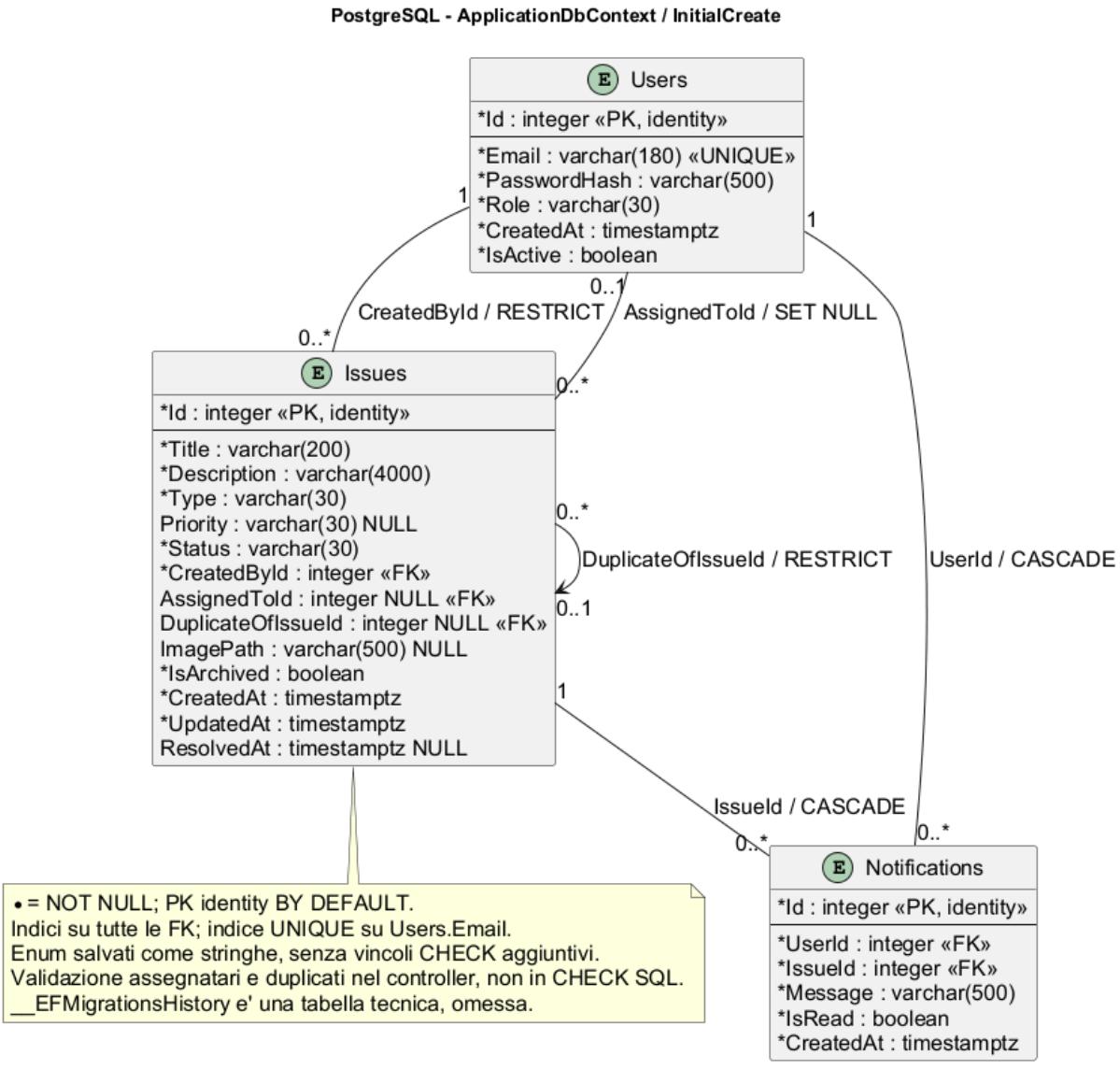


Figura 2.5: Schema della persistenza.

2.4 Autenticazione e autorizzazione

Il login normalizza l'email, verifica account attivo e password con `Pbkdf2PasswordHasher`, quindi restituisce JWT, scadenza e identita' dell'utente. Il frontend conserva la sessione in localStorage, ne controlla la scadenza e invia il token solo alle API del progetto. Un 401 su una richiesta protetta porta al login; un 403 non comporta logout.

I guard e i pulsanti nascosti migliorano l'interfaccia, ma la sicurezza dipende dalle autorizzazioni backend. ADMIN crea utenti, archivia e marca duplicati. USER cambia stato soltanto per le proprie assegnazioni; READONLY non modifica issue. Le notifiche sono consultabili soltanto dal destinatario, anche rispetto ad ADMIN. Secondo la decisione del gruppo su RF-17, solo USER e ADMIN possono segnare lette le proprie notifiche; READONLY puo' soltanto consultarle. Il rifiuto server e i comandi UI coerenti con questa decisione sono verificati nelle evidenze del 19 ottobre. Il link Utenti e' mostrato solo ad ADMIN; la rotta resta protetta anche con navigazione diretta.

Il setup locale usa HTTP e credenziali di sviluppo. Un'eventuale esposizione pubblica richiederebbe HTTPS e configurazione adeguata dei segreti; questo documento non certifica un deployment di produzione. Connessione, chiave JWT e credenziali di bootstrap non sono incorporate nei file appsettings. Compose passa i valori dimostrativi; l'avvio nativo richiede variabili d'ambiente o user-secrets. Il seeder non modifica un ADMIN gia' presente. L'API rifiuta enum numerici non definiti per tipo, priorita' e stato; la priorita' omessa resta valida.

2.5 Ricerca, suggerimento ed export

La lista esclude gli archiviati e applica filtri per tipo, stato, priorita', creatore e assegnatario dove previsti dall'API. La ricerca API usa `ILike` di PostgreSQL su titolo e descrizione. L'helper `SearchIssues` opera invece su collezioni in memoria: i suoi test non sostituiscono i test HTTP della query SQL.

Il suggerimento considera gli utenti attivi ADMIN e USER. Conta soltanto issue assegnate non archiviate in TODO o IN_PROGRESS. Vince il carico minore; a parita', l'ID minore. Nessun candidato produce 404 con messaggio. La consultazione del suggerimento non scrive dati.

Lista ed export riutilizzano la stessa query backend. Il CSV contiene Id, Title, Type, Priority, Status, CreatedByEmail, AssignedToEmail e CreatedAt; usa UTF-8 ed escaping di virgole, doppi apici e ritorni a capo. Il frontend scarica il Blob con nome `issues.csv` e passa i filtri correnti anche durante l'attesa della ricerca live. RxJS `switchMap` annulla le richieste precedenti evitando risultati obsoleti. Il download browser aggiunge BOM UTF-8 e la dichiarazione `sep=,` per l'apertura in Excel; la risposta CSV standard dell'API resta invariata.

2.6 Immagini, archivio e duplicati

L'immagine viene caricata dopo la creazione come multipart con campo `image`. Sono ammessi PNG/JPEG non vuoti entro 5 MiB (5242880 byte). Il backend controlla estensione e marcatori del file, non esegue una decodifica completa o una scansione antivirus. Solo ADMIN o USER creatore puo' caricare; tutti i ruoli autenticati possono leggere. L'immagine viene scaricata tramite HttpClient con JWT e visualizzata con un object URL, revocato quando la pagina viene distrutta.

Un upload fallito non annulla la creazione: il form conserva l'ID e riprova solo il caricamento. L'archiviazione imposta `IsArchived`, senza cancellazione; l'archivio mantiene accessibili detta-

glio e immagine. Il duplicato imposta DUPLICATE e `DuplicateOfIssueId`, azzera `ResolvedAt` e non genera la notifica di risoluzione.

2.7 Diagrammi comportamentali

Le viste seguenti descrivono creazione, cambio stato e duplicazione.

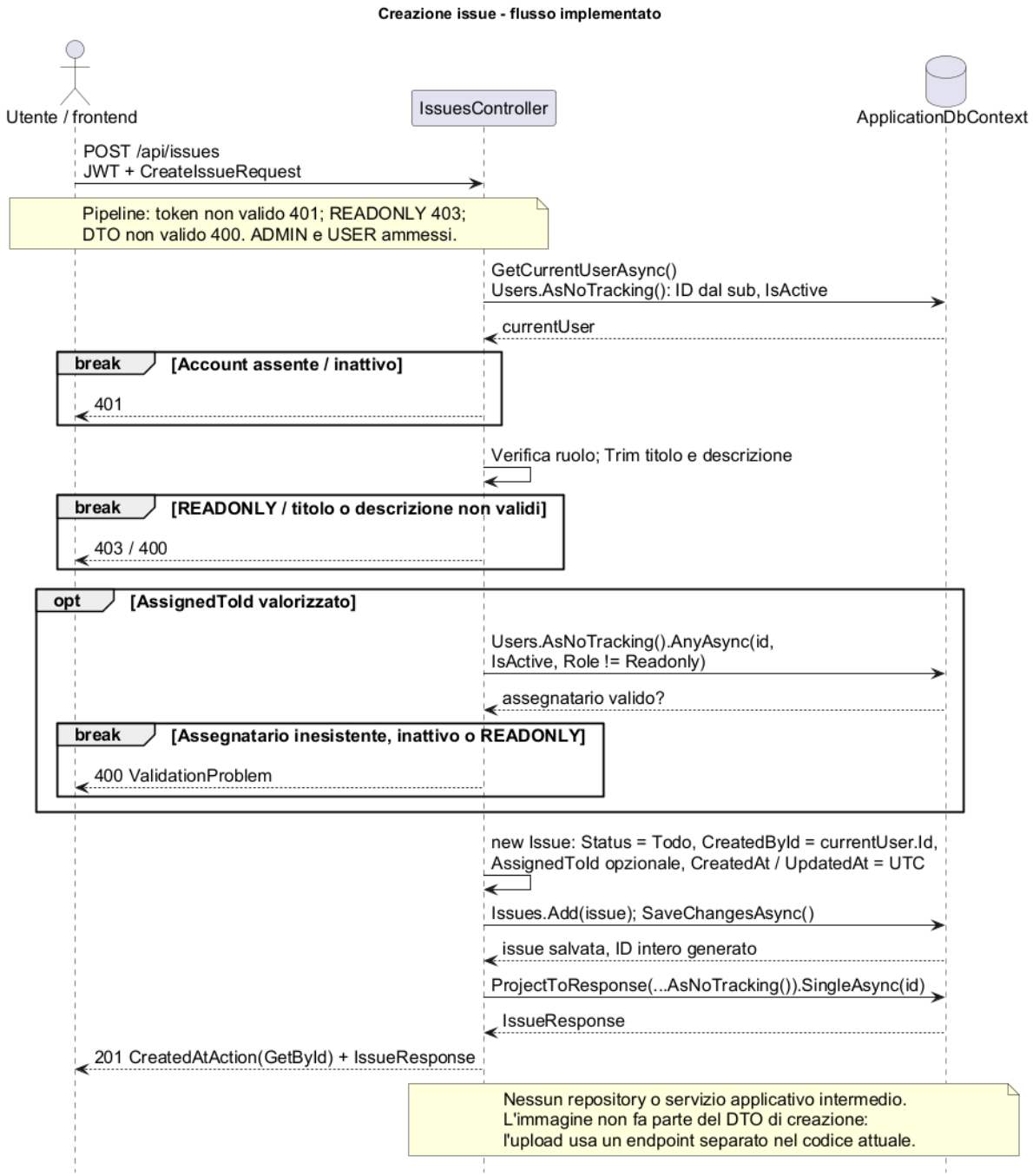


Figura 2.6: Sequenza di creazione della issue.

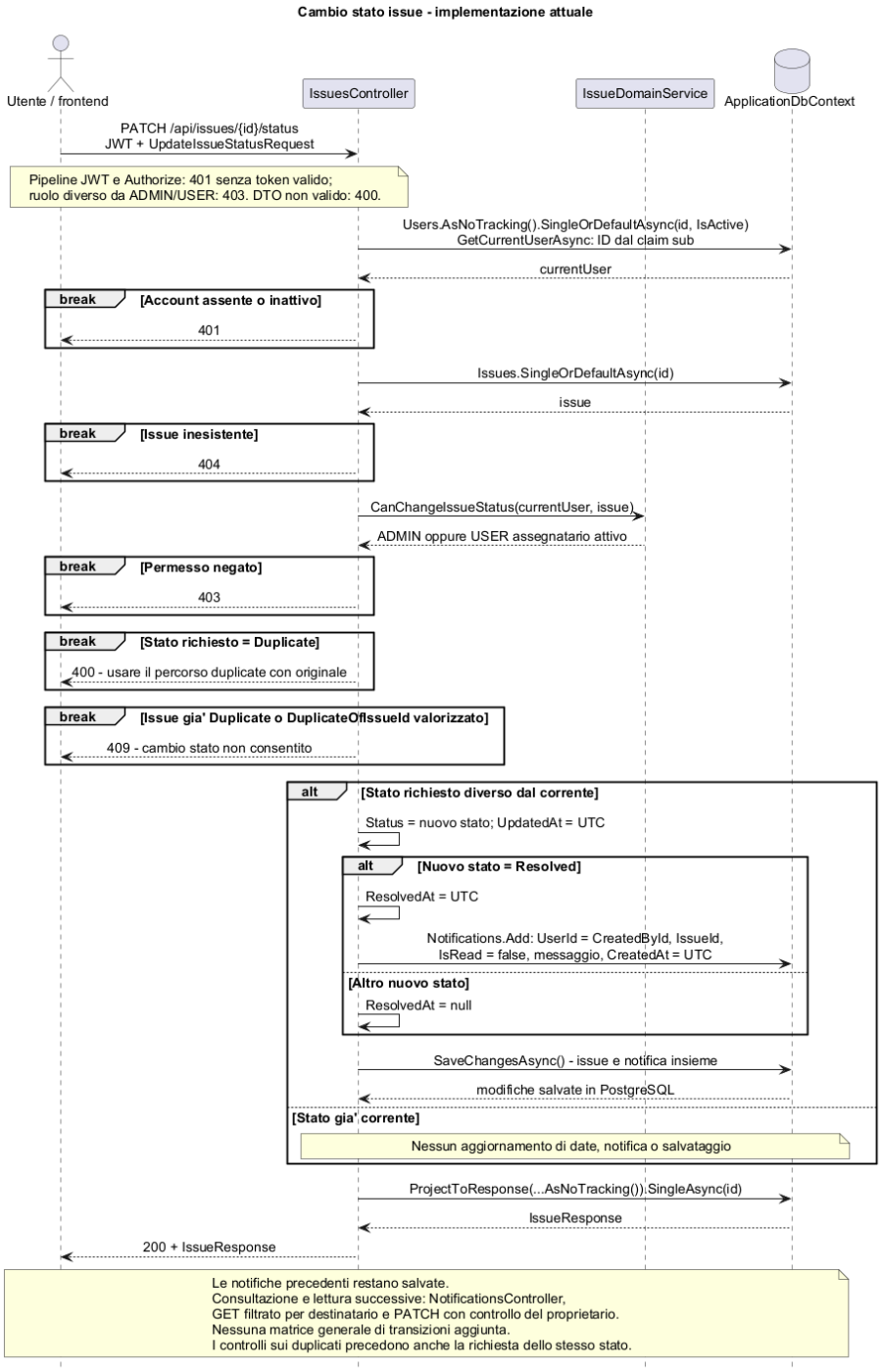


Figura 2.7: Sequenza di cambio stato e notifica.

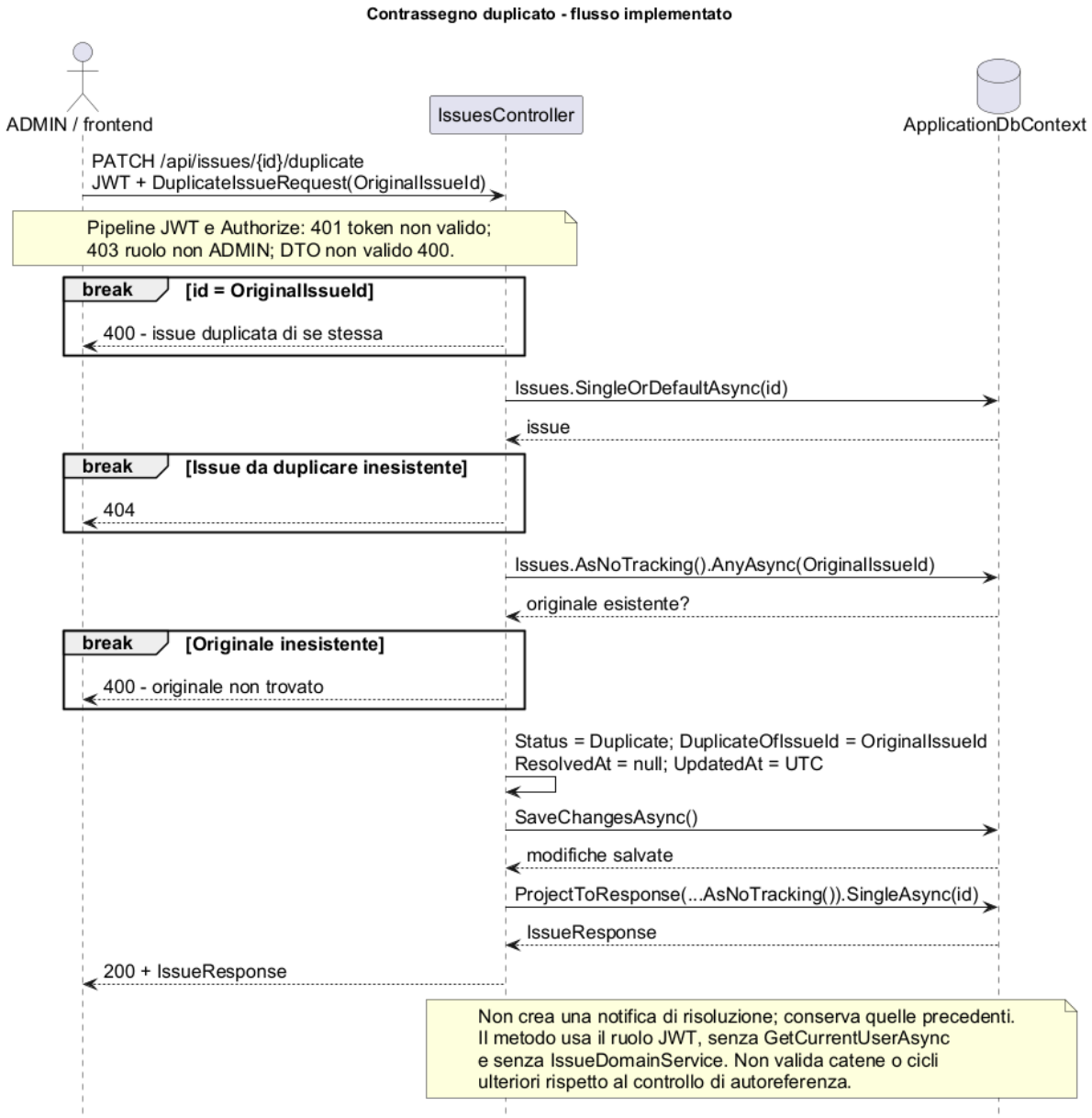


Figura 2.8: Sequenza di contrassegno duplicato.

2.8 Interfacce e avvio

Metodo	Percorso API	Scopo
POST	`/api/auth/login`	Login.
GET / POST	`/api/users`	Lista/creazione utenti, solo ADMIN.
GET / POST	`/api/issues`	Lista/creazione issue.
GET	`/api/issues/{id}`	Dettaglio.
PATCH	`/api/issues/{id}/status`	Cambio stato.
PATCH	`/api/issues/{id}/archive`	Archiviazione ADMIN.
GET	`/api/issues/archived`	Archivio.
PATCH	`/api/issues/{id}/duplicate`	Contrassegno duplicato ADMIN.
GET	`/api/issues/suggest-assignee`	Suggerimento.
GET	`/api/issues/export`	CSV.
POST / GET	`/api/issues/{id}/image`	Upload/lettura immagine.
GET	`/api/notifications`	Notifiche personali.
PATCH	`/api/notifications/{id}/read`	USER/ADMIN segnano lette le proprie notifiche; READONLY riceve 403.

Le rotte frontend sono `/login`, `/issues`, `/issues/new`, `/issues/:id`, `/issues/archived`, `/users` e `/notifications`. La pagina utenti e' riservata ad ADMIN; la creazione e' protetta anche contro l'accesso diretto READONLY.

Prerequisiti: Docker Desktop con motore Linux, Node.js 22/npm; .NET SDK 10 per i test o per l'avvio backend fuori da Docker. Dalla radice:

```
docker compose -f docker/docker-compose.yml up -d --build
# In un altro terminale:
cd frontend
npm ci
npm start
```

Frontend: `http://localhost:4200/login`; API: `http://localhost:5256/api`. Attendere l'avvio delle API prima del login. Su database nuovo l'ADMIN e' `admin@bugboard26.local`, password `Admin123!`: sono credenziali locali di sviluppo. Il README contiene le alternative di configurazione. Non usare `down -v` per conservare i volumi. L'avvio con `dotnet run` usa una directory immagini diversa dal container e non trasferisce automaticamente i file. L'avvio nativo richiede connessione PostgreSQL, chiave JWT e credenziali di bootstrap configurate tramite ambiente o User Secrets, come descritto nel README; non sono conservati segreti di avvio nei file applicativi versionati.

2.9 Versionamento e artefatti

Il repository Git indicato all'inizio raccoglie codice, test e documenti. La cronologia Git documenta l'evoluzione dei sorgenti e i contributi; il riferimento della revisione descritta e' riportato in copertina. Sorgenti PlantUML e PNG correnti sono verificati insieme; i file VPD sono conservati come artefatti storici. Provenienza delle viste e limiti della verifica CASE sono descritti nel capitolo 4.

Capitolo 3

Testing e valutazione dell'usabilita'

3.1 Strategia e livelli di verifica

Le verifiche sono separate in test di dominio, integrazione backend e collaudo frontend con sistema reale. I test di dominio usano xUnit e collezioni in memoria. I test HTTP backend usano WebApplicationFactory con autenticazione reale e PostgreSQL dedicato; non richiedono una porta API esterna. Ogni caso usa un database temporaneo distinto. Il database di sviluppo non viene svuotato dai test.

Il frontend usa Jasmine/Karma. I suoi test simulano servizi o HTTP per isolare il comportamento dei componenti: un test passato non dimostra da solo il collegamento al backend. Il collaudo del 16 ottobre ha invece usato Angular, backend Docker e PostgreSQL reali, con Chrome headless.

3.2 Tre metodi di dominio e classi di equivalenza

Metodo	Casi presenti e motivazione
`SearchIssues` (issues, keyword)	Parola nel titolo, nella descrizione e parola assente: distingue i due campi di ricerca e la classe senza corrispondenze.
`CanChangeIssueStatus` (user, issue)	ADMIN non assegnatario; USER assegnatario/non assegnatario; READONLY anche assegnato; USER inattivo. Separa ruolo, assegnazione e attivazione.
`SuggestAssignee` (users, issues)	Carichi differenti con vincitore alternato, issue risolte/chiuse escluse e nessun candidato assegnabile. Verifica la scelta e il caso limite senza risultato.

I tre metodi hanno ciascuno due parametri. Le Theory rappresentano casi significativi rispetto alle regole del dominio. La suite di integrazione aggiunge verifiche di permessi, filtri, escaping CSV, parita' ed esclusioni del suggerimento, immagini e persistenza. La copertura strutturale completa non e' deducibile da questo elenco: va misurata sulla suite completa.

3.2.1 Classi coperte e limiti della strategia

Riferimenti: `backend/BugBoard26.Api/Services/IssueDomainService.cs` e `backend/BugBoard26.Tests/IssueDomainServiceTests.cs`. La strategia usa classi di equivalenza e

regole decisionali; non costituisce una suite completa SECT/R-WECT e non dimostra copertura strutturale completa.

SearchIssues. Le keyword presenti sono `login`, `API` e `grafica`: coprono titolo, descrizione e assenza di risultati. Il test asserisce il conteggio, non gli ID restituiti. Keyword nulla, vuota, di soli spazi, trimming, differenze di maiuscole, elenco vuoto e molteplici corrispondenze non hanno casi dedicati in questo file. In particolare, la keyword API e' identica al testo della fixture: non prova da sola il confronto case-insensitive del metodo. Le prove PostgreSQL della ricerca API restano distinte da quelle dell'helper in memoria.

CanChangeIssueStatus. Le classi coperte separano ADMIN attivo, USER assegnatario/non assegnatario, READONLY assegnatario e USER inattivo. L'oracolo e' il booleano atteso. Non sono coperte tutte le combinazioni: per esempio ADMIN inattivo e USER attivo con assegnatario nullo non hanno casi unitari dedicati. Date, transizioni effettive e salvataggi appartengono al controller e richiedono test HTTP/persistenza, non questo metodo puro.

SuggestAssignee. Le Theory usano carichi (0,2) e (3,1) e verificano l'ID del vincitore; le fixture escludono un READONLY e issue Resolved/Closed. Il Fact usa soltanto READONLY attivo e USER inattivo e attende `null`. Parita' per ID, ADMIN candidato, InProgress, archiviati ed elenco vuoto non hanno casi unitari dedicati nel file. Il codice implementa queste regole e sono presenti scenari HTTP aggiuntivi; non si trasforma la loro presenza in copertura unitaria misurata. La suite del 19 ottobre esegue anche questi test HTTP: 166 casi complessivi superati, 11 di dominio e 155 di integrazione. Il report e il TRX sono in `verifica-backend/`; non si attribuiscono alle sole prove unitarie i risultati ottenuti attraverso l'API.

3.3 Test plan – Risoluzione di una issue

Obiettivo: verificare UC-02 e RF-10/RF-11 senza consentire modifiche a utenti non autorizzati. Il piano prevede una issue di prova creata da S e assegnata ad A, entrambi USER attivi e distinti; U e' un altro USER, R e' READONLY, con un ADMIN separato. Stato, date e notifiche vengono confrontati prima e dopo le operazioni, usando dati predisposti per la prova.

3.3.1 Ambiente, criteri e raccolta delle evidenze

Ambito: UC-02, RF-10, RF-11, RF-17, RNF-02 e feedback UI di RNF-05. Sono esclusi da questo piano export, immagini e archivio, verificati separatamente nei collaudi descritti nella relazione. Ambiente previsto: checkout stabile, .NET 10, xUnit e PostgreSQL 16 dedicato ai test sulla porta 55432. Il controllo dal browser richiede inoltre Angular e l'API del setup di prova. Gli account e gli ID sono predisposti nelle fixture, senza riportare token o credenziali nei risultati. Nei casi rifiutati viene confrontato uno snapshot prima/dopo; account inattivo e issue duplicata hanno dati dedicati.

Ingresso: disponibilita' di data, riferimento del codice e modifiche locali, versioni, ambiente, comando o procedura. API, login e database isolato devono essere disponibili. Un prerequisito assente rende il caso bloccato, non superato. La suite HTTP usa database distinti per caso e JWT ottenuti dal login reale. Una fixture puo' predisporre condizioni non ottenibili dalla UI, per esempio un READONLY assegnatario, senza modificare i dati di sviluppo.

Oracoli: confronto di stato, date UTC, destinatario e notifiche persistenti. Due chiamate separate a `UtcNow` non devono necessariamente produrre date identiche. Stesso stato valido: date e notifiche restano invariate; rifiuto: nessuna scrittura. Sono registrati risposta HTTP, riscontro persistente, risultato effettivo ed evidenza. Il proprietario viene verificato anche quando chi tenta la lettura e' ADMIN.

Evidenze acquisite: suite backend, coverage, persistenza e analisi qualita' sono in `verifica-backend/` sotto Documentazione finale. Il report frontend storico e' in `verifica-frontend/`; l'audit del 19 ottobre registra i nuovi esiti. RF-17 e' verificato in backend e UI. Il CSV associa i casi ai metodi passati nel TRX e ai riscontri browser, distinti dalle sessioni di usabilita' con persone.

La tabella comprende i casi TP-01–TP-14. I riferimenti ai test sono: stato e permessi in `IssueStatusIntegrationTests.cs`; duplicati in `IssueDuplicateValidationIntegrationTests.cs`; notifiche in `NotificationsIntegrationTests.cs`. Riapertura/seconda risoluzione, account disattivato e ripetizione del PATCH read hanno ora esiti dedicati nel TRX corrente. Per TP-14 la decisione del gruppo RF-17 fissa l'oracolo: GET delle proprie notifiche 200; PATCH read su notifica propria esistente 403, anche se gia' letta, senza modificare `IsRead`, date o altre righe. Le fixture predispongono per R notifiche lette e non lette. Il collaudo verifica l'assenza del comando UI e di PATCH automatici durante la consultazione; USER e ADMIN mantengono la lettura delle proprie notifiche (TP-13 e regressione per ADMIN). Esiti e riferimenti sono nel CSV; il criterio RF-17 e' invariato.

Caso	Azione	Risultato atteso
TP-01	A imposta RESOLVED.	200; stato e date persistiti; nuova notifica non letta per S.
TP-02	ADMIN non assegnatario aggiorna una issue di prova.	Operazione consentita.
TP-03	U tenta il cambio stato.	Nessun pannello frontend; PATCH diretto 403 senza scritture.
TP-04	R tenta il cambio stato.	Nessun pannello; PATCH diretto 403.
TP-05	Richiesta senza JWT.	401 e nessuna modifica.
TP-06	ID issue inesistente.	404 e nessuna notifica.
TP-07	A ripete lo stato RESOLVED corrente.	Nessuna nuova notifica o variazione delle date.
TP-08	Riapertura e seconda risoluzione.	Data azzerata alla riapertura; nuova data/notifica alla seconda risoluzione.
TP-09	Cambio stato su duplicata.	409; riferimento e stato conservati.
TP-10	S consulta notifiche; U tenta lettura della notifica di S.	S vede la propria notifica; U non la vede e PATCH read altrui restituisce 403.
TP-11	JWT di account diventato inattivo.	401 dal lookup dell'account; nessuna modifica persistente.
TP-12	Si richiede Duplicate nel percorso status.	400; stato, riferimento, date e notifiche invariati.
TP-13	S segna letta la notifica e ripete il PATCH.	200; lettura persistita; nessuna notifica aggiunta; ID assente 404, senza token 401.
TP-14	READONLY consulta le proprie notifiche e tenta di segnarne una letta.	GET 200, solo notifiche proprie; PATCH read 403 e dati invariati. UI senza comando di lettura. Decisione del gruppo RF-17 verificata.

Criterio di accettazione: tutti i casi pianificati eseguiti con esito registrato e nessuna modifica non autorizzata. Gli esiti effettivi sono in `esiti_test_plan.csv`, con rimandi a metodi di test e TRX; i controlli UI richiamano il collaudo browser. TP-08 e TP-11 hanno casi HTTP dedicati;

TP-13 include il PATCH ripetuto e TP-14 il rifiuto READONLY anche su notifiche gia' lette. Gli esiti dei test tecnici sono distinti dalle sessioni di usabilita' con persone.

3.4 Collaudo frontend reale – 16 ottobre 2026

Il report del collaudo e gli screenshot sono in `verifica-frontend/report.md` e `verifica-frontend/immagini/`. Riferimento: commit `e13995140dff382dc3d548208c6c1e60be91fe5d`, checkout pulito prima e dopo. API sulla porta 5256 e Angular sulla 4200; PostgreSQL del Compose di sviluppo. Le prove hanno utilizzato il sistema avviato, senza sostituire le risposte delle API con simulazioni.

Sono stati creati tre account con prefisso `collaudo-20261016084339` (due USER e un READONLY) e le issue 4–7. Le operazioni di modifica hanno riguardato soltanto queste nuove issue. La issue 4 e' stata duplicata e archiviata; la 7, predisposta tramite API per avere un assegnatario deterministico, e' stata risolta dal relativo USER. Le altre creazioni sono passate dal form frontend. Non sono riportati token o credenziali dei partecipanti nei risultati.

Area	Esito osservato
Login e guard	Login ADMIN riuscito; password errata: 401 e messaggio UI. Accesso anonimo a creazione reindirizzato al login; nessuna navbar nella login.
Ricerca e lista	Ricerca live, digitazione rapida con risultato finale corretto, filtri tipo/stato/priorita', ordinamento per titolo, lista vuota e reset completo riusciti.
Export	Download reale; ID e ordine nel CSV uguali alla lista filtrata.
Creazione	Issue senza immagine e con PNG create; immagine scaricata autenticata e visualizzata.
Recupero upload	JPEG volutamente non valido rifiutato dall'API; ID conservato; JPEG valido caricato al secondo tentativo senza seconda creazione.
Validazione	Titolo/descrizione di soli spazi: nessuna POST e messaggi sui campi. Formato non ammesso e file oltre 5 MiB rifiutati dal frontend.
Stato e ruoli	USER assegnatario risolve; stato persistito. USER non assegnatario senza pannello. USER non assegnatario e READONLY ricevono 403 dal PATCH diretto.
READONLY	Nessuna azione di modifica nel dettaglio; accesso diretto a creazione reindirizzato; immagine consultabile.
Duplicato e archivio	ADMIN marca duplicato con collegamento aggiornato; archivia; issue assente dalla lista principale e dettaglio apribile dall'archivio.
Errori e layout	ID inesistente mostra "Issue non trovata". Lista, form e dettaglio controllati a 1366 e 390 pixel senza overflow orizzontale; screenshot ispezionati.
Notifica	Via API verificata la notifica di risoluzione della issue 7 al segnalatore ADMIN; non un collaudo completo della pagina notifiche.

La build frontend e' riuscita e i 27 test Jasmine/Karma sono passati senza watch con ChromeHeadless. La prova non ha richiesto correzioni applicative. Gli screenshot selezionati sono conservati in `verifica-frontend/immagini/`.

Questa prova non misura le prestazioni con rete rallentata o grandi quantita' di dati, non copre ogni browser e non sostituisce il collaudo di persistenza dopo ricreazione dei container.

3.5 Esecuzione ripetibile e qualita' backend

Le verifiche successive sono registrate separatamente in `verifica-backend/` e `verifica-frontend/review-20261019/`. I report includono comandi, data, HEAD, modifiche non committate e SHA256. Il 19 ottobre sono stati eseguiti 166 casi backend (11 dominio, 155 integrazione) con PostgreSQL dedicato: tutti superati, nessuno ignorato. Il test plan e' ricondotto ai singoli metodi nel CSV degli esiti.

Il collaudo integrato dell'audit del 19 ottobre ha verificato 20 controlli con API Docker reali e database E2E dedicato, senza mock HTTP: utenti, creazione con PNG, suggerimento, dettaglio, risoluzione, notifiche USER/ADMIN, RF-17, CSV, duplicazione e archivio. Le correzioni UI successive riguardano la visibilita' del link Utenti e il nome accessibile della ricerca; il report di audit registra la verifica successiva. Le prove con utenti restano distinte.

Dalla radice del repository, con Docker disponibile:

```
docker compose -p bugboard26-tests ‘
  -f backend/BugBoard26.Tests/docker-compose.tests.yml up -d --wait
dotnet test backend/BugBoard26.slnx
# Frontend, dalla cartella frontend:
npm run build
node node_modules/@angular/cli/bin/ng.js test ‘
  --watch=false --browsers=ChromeHeadless
```

Il database di test usa la porta 55432 ed e' distinto da quello di sviluppo. La sola esecuzione dei test di dominio con filtro non sostituisce la suite di integrazione. Dopo l'adattamento del download CSV i test Jasmine/Karma sono 40, inclusi controlli di navigazione e creazione per ruolo e verifica dei byte esportati, eseguiti in ChromeHeadless senza watch. La build e' riuscita e il browser reale ha verificato il download. Il resoconto del collaudo interno riporta inoltre l'apertura corretta del CSV in Excel.

SonarQube locale e la raccolta OpenCover sono predisposti in `quality/analyze-backend.ps1` e `quality/coverage.runsettings`. L'analisi corrente e' stata elaborata dal server locale Community Build 26.10.0.133078, scanner .NET 11.3.0: coverage importata per 29 file principali, Quality Gate OK con condizioni valutate, zero bug e zero vulnerabilita' segnalati, quattro code smell lasciati visibili. Coverage Sonar complessiva 93,7%, linee 95,7%, rami 88,2%. Non si dichiara copertura totale o assenza di ogni possibile difetto. Gate e percentuali sono riferiti a HEAD con le correzioni locali, non al solo commit `6aa175a`.

Tre code smell S6932 riguardano la lettura manuale del multipart, mantenuta per preservare autorizzazioni e risposte 400 uniformi; un avviso ASP0027 riguarda Program pubblico parziale. Non sono introdotte esclusioni o soppressioni. L'avviso storico su Microsoft.OpenApi 2.0.0 e' superato dal pin 2.7.5. Dati e immagini sono stati confrontati dopo restart e ricreazione del solo backend, senza eliminare volumi; il rebuild successivo conserva la stessa issue e lo stesso SHA256 immagine. Nessun test automatico usa il database di sviluppo; le prove Docker con dati riconoscibili sono distinte.

3.6 Valutazione dell'usabilita' – review e sessioni

La valutazione comprende una review tecnica sistematica e tre sessioni con partecipanti potenziali, documentate dal resoconto del moderatore e dal questionario. I due metodi sono complementari: il collaudo tecnico verifica comportamenti del sistema, mentre le sessioni raccolgono riscontri d'uso. I soli voti non permettono di ricostruire tempi, attivita' non osservate o cause delle difficolta'.

3.6.1 Review tramite checklist

La review sistematica assistita del 19 ottobre e' registrata in `evidenze/review_tecnica.csv`, con otto criteri, evidenze e limiti. Il report completo e' in `verifica-frontend/review-20261019/report.md`; la prima review backend resta storica. Sono ispezionati sorgenti e screenshot e provati 28 controlli browser con API Docker reale e PostgreSQL dedicato. La review e' assistita da automazione browser e non costituisce una valutazione umana indipendente o una certificazione WCAG.

- Terminologia coerente fra lista, form e dettaglio.
- Navigazione riconoscibile e ritorno alla lista disponibile.
- Campi obbligatori ed errori comprensibili vicino al campo.
- Caricamento, esito del salvataggio, errore e lista vuota distinguibili.
- Azioni coerenti con il ruolo, senza pulsanti di modifica per READONLY.
- Riconoscibilita' della notifica di risoluzione e dello stato letto/non letto.
- Uso da tastiera, focus visibile e leggibilita' su schermi piccoli.
- Recupero dell'upload senza indurre una seconda creazione.

Login, ricerca e creazione sono percorsi con Tab, Shift+Tab ed Enter, controllando il focus visibile. Sono distinti caricamento, lista vuota e errore di rete; ritardo e interruzione sono simulati soltanto nel browser, con recupero tramite Pulisci. Un falso PNG e' invece realmente rifiutato dall'API: il form mostra l'ID gia' salvato e il nuovo upload valido usa lo stesso ID, con una sola richiesta di creazione. Nessun dato di sviluppo e' modificato. Quattro viste sono controllate a 320 e 390 pixel senza overflow orizzontale rilevato.

Quattro dei cinque campioni di contrasto della lista erano insufficienti e sono corretti: intestazioni e nota export passano a 4,97:1, priorita' a 5,25:1, stato a 5,63:1; il pulsante resta a 4,70:1. Le soglie sono confrontate sui valori non arrotondati. Sono scurite anche note analoghe del form/dettaglio e testo di errore login, senza ridisegnare l'interfaccia. I 40 test frontend esistenti e la build sono superati nuovamente; non sono aggiunti test permanenti. Restano migliorabili annunci per lettori schermo, focus iniziale sugli errori e conferma delle azioni admin. La review e' completata nel perimetro dichiarato, non e' una certificazione di accessibilita'.

3.6.2 Sessioni con utenti e questionario

Il protocollo definisce cinque attivita': ricerca di una issue (U1), creazione (U2), risoluzione come assegnatario (U3), consultazione e lettura della notifica (U4), filtro ed export in sola lettura (U5). Nelle tre sessioni registrate sono state svolte soltanto U2 e U4; non si attribuisce ai partecipanti l'esecuzione dell'intero protocollo.

Il questionario usa una scala da 1 (per nulla) a 5 (molto) per navigazione, comprensibilita' dei campi, ricerca, esiti delle operazioni e permessi. Prevede inoltre una domanda sul passaggio meno

chiaro e un suggerimento facoltativo. Per queste due domande non sono disponibili risposte. Si tratta di un questionario locale breve, non di uno strumento SUS. Protocollo e testi delle attivita' sono in `usabilita.md`; sessioni e punteggi sono conservati in `evidenze/sessioni_utenti.csv` e `evidenze/questionario.csv`.

3.6.3 Risultati e limiti delle tre sessioni

Fonte: resoconto del moderatore e risposte trascritte del questionario; non sono allegate registrazioni delle sessioni. Data: 19 ottobre 2026. Il campione di convenienza comprende tre persone conosciute dal moderatore, che hanno usato il suo PC in condivisione, tutte con ruolo USER. P01 e P03 hanno svolto U2, creazione issue; P02 U4, riscontro e lettura notifica. Per tutti il completamento e' dichiarato autonomo dopo una spiegazione iniziale, che viene esplicitamente registrata. Non e' descritto il contenuto dell'introduzione o il mezzo di condivisione.

Codice	Attivita'	Q1	Q2	Q3	Q4	Q5
P01	U2	5	4	4	4	5
P02	U4	4	4	5	4	4
P03	U2	4	3	4	4	3

Q1 indica navigazione, Q2 campi, Q3 ricerca, Q4 esiti, Q5 permessi; scala da 1 (per nulla) a 5 (molto). Sono riportati i valori ricevuti senza trasformarli in un punteggio SUS o in una validazione statistica. I voti 3 di P03 per campi e permessi indicano una valutazione moderata: in assenza di osservazioni non si attribuiscono cause specifiche.

I partecipanti sono identificati con codici senza nomi nei file consegnati; il moderatore li conosce, quindi non si dichiara anonimato assoluto. Campione di convenienza ridotto, attivita' diverse e solo ruolo USER limitano la generalizzabilita'. Non si assume che ciascuno abbia svolto ricerca, filtri o prove con gli altri ruoli. Profilo, esperienza, browser, checkout effettivo, tempi e risposte aperte Q6/Q7 non sono comunicati. I campi vuoti significano non fornito, non assenza di problemi. La modalita' del consenso non e' documentata. La review tecnica resta un'attivita' distinta. Dettagli e provenienza: `evidenze/risposte-parziali-usabilita.md`.

Capitolo 4

Limiti e conclusioni

4.1 Limiti della valutazione

Le verifiche tecniche sono riferite alle revisioni e agli ambienti identificati nei report. Non coprono tutti i browser, condizioni di carico elevato o un deployment pubblico. Il superamento dei test e il Quality Gate non rappresentano una certificazione completa di sicurezza o prestazioni. Le sessioni di usabilita' sono limitate a tre partecipanti, al ruolo USER e a due attivita'; le lacune del resoconto sono indicate nel capitolo 3.

4.2 Provenienza dei diagrammi e verifica CASE

I diagrammi riportati nella relazione sono generati dai sorgenti UML PlantUML versionati nella repository. La revisione documentale del 16 ottobre registra PlantUML 1.2026.8 e Java 21 come ambiente di generazione. Il manifest `diagrammi_manifest.json` riporta percorsi e impronte dei file; il controllo confronta i sorgenti con i metadati incorporati nei PNG. Questo riscontro documenta la coerenza fra sorgente ed esportazione.

I file Visual Paradigm sono conservati come artefatti storici. Non e' documentata la sincronizzazione dei modelli nativi con tutte le viste attuali. La coerenza dei sorgenti PlantUML e dei PNG non dimostra, da sola, l'adempimento del vincolo sull'uso di strumenti CASE della sezione 3.2 della traccia. Tale conformita' resta quindi un punto non attestato dalle verifiche raccolte, distinto dalla correttezza dei comportamenti applicativi sottoposti a test.

4.3 Perimetro del prodotto

Non sono previsti paginazione, ripristino archivio, refresh token o gestione immagini dal dettaglio frontend. Queste esclusioni mantengono il prodotto nel perimetro assegnato.

4.4 Conclusioni

BugBoard26 realizza le funzionalita' assegnate con frontend e backend indipendenti, dati centralizzati e operazioni regolate dai ruoli. Test di dominio, integrazione, collaudi browser e analisi statica supportano i comportamenti verificati. La review ha portato a correzioni puntuali dell'interfaccia e le sessioni con partecipanti hanno fornito un primo riscontro d'uso. Questi risultati restano circoscritti ai metodi, agli scenari e ai limiti dichiarati nella relazione.